In this case, `array` will have 25 entries, because `square` is declared `constexpr`. Of course, just because the function *can* be used in a constant expression doesn't mean that all uses are automatically constant expressions:

```
int dummy=4;
int array[square(dummy)];
```

❶ Error, dummy is not a constant expression

In this example, `dummy` is not a constant expression ❶, so `square(dummy)` isn't either—it's just a normal function call—and thus can't be used to specify the bounds of `array`.

A.4.1. constexpr and user-defined types

Up to now, all the examples have been with built-in types such as `int`. However, the new C++ Standard allows constant expressions to be of any type that satisfies the requirements for a *literal type*. For a class type to be classified as a literal type, the following must all be true:

- It must have a trivial copy constructor.
- It must have a trivial destructor.
- All non-`static` data members and base classes must be trivial types.
- It must have either a trivial default constructor or a `constexpr` constructor other than the copy constructor.

We'll look at `constexpr` constructors shortly. For now we'll focus on classes with a trivial default constructor, such as class `CX` in the following listing.

Listing A.3. A class with a trivial default constructor